

ServerKit

OOP-First Linux Operations SDK
with Interactive Shell and Local AI

Master Project Document · Two-Developer Build

1. Executive Summary

ServerKit is an object-oriented Python SDK that abstracts Linux resources into clean Python objects and fluent, chainable workflows. Rather than memorising shell commands and piping text output, developers interact with structured objects.

Think: Pandas for Linux operations. Instead of raw text streams from `ps aux` or `grep`, you get Python objects you can filter, sort, chain, save, and replay.

Three-Layer Architecture

Layer	Responsibility
SDK (Core)	The primary product. Processes, Logs, Workflows as Python objects.
Interactive Shell	A productivity frontend for the SDK. Exploration and discovery.
AI Layer	Optional assistant. Translates natural language into SDK actions via local Ollama.

What ServerKit Is NOT

- A Linux replacement
- An AI agent that runs arbitrary commands
- A shell replacement
- A command generator

What ServerKit IS

- A structured, object-oriented abstraction over Linux resources
- A workflow engine for reusable operational investigations
- A teaching project demonstrating real-world OOP patterns

2. Problem Statement

Linux administration requires remembering commands, chaining tools, parsing text, and writing one-off shell scripts. Typical examples:

```
ps aux | grep python
cat app.log | grep ERROR
find . -name '*.py'
```

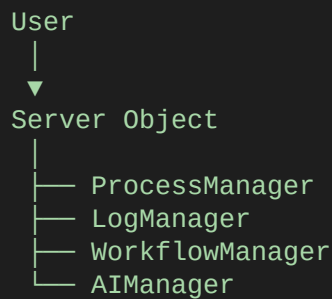
```
df -h
```

These commands are:

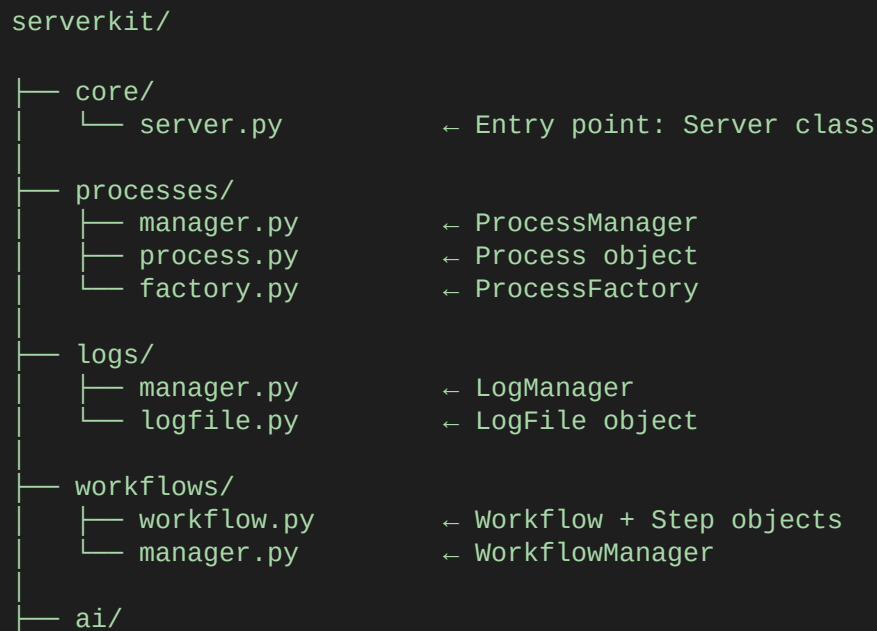
- Difficult to compose programmatically
- Difficult to serialize and reuse
- Difficult to extend or test
- Unstructured text output — hard to process further

ServerKit introduces a structured, object-oriented abstraction layer that solves all of these.

3. Full Architecture



Project Structure



```

|   ├── analyzer.py      ← Diagnostic AI
|   ├── ollama_client.py ← Ollama API wrapper
|
|   ├── shell/
|   |   ├── repl.py      ← Interactive shell
|   |   ├── autocomplete.py ← SDK-aware autocomplete
|   |   └── parser.py    ← Input parser
|
|   ├── tests/
|   └── examples/

```

OOP Patterns Used

Pattern	Where Used
Composition	Server holds ProcessManager, LogManager, WorkflowManager, AIManager
Encapsulation	psutil and raw syscalls hidden behind clean objects
Builder Pattern	server.workflow().processes().memory_above(1000).sort_by_memory()
Factory Pattern	ProcessFactory.create(raw_process)
Fluent Interface	Each method returns self for chaining
Strategy Pattern	SequentialExecutor / ParallelExecutor for workflow execution
Composite Pattern	Workflow composed of Step objects

4. SDK Modules (Core)

Processes

All process operations start from `server.processes()`. Each method filters the collection and returns self for chaining.

```

server.processes().all()
server.processes().named('python')
server.processes().memory_above(500)
server.processes().cpu_above(50)
server.processes().sort_by_memory()
server.processes().sort_by_cpu()

```

Process Object:

```
process.pid          # int
process.name         # str
process.memory_mb    # float
process.cpu_percent  # float

process.kill()
process.terminate()
process.details()
```

Logs

```
server.logs('app.log').errors()
server.logs('app.log').warnings()
server.logs('app.log').contains('timeout')
server.logs('app.log').tail(100)
server.logs('app.log').summarize()
```

Workflows

Workflows are reusable, serializable investigation pipelines.

```
workflow = (
    server.workflow('memory_audit')
        .processes()
        .memory_above(1000)
        .sort_by_memory()
        .summarize()
)

workflow.save()
server.run('memory_audit')
```

5. Workflow System

Workflows are a first-class feature — not an add-on. They justify the Builder Pattern, serialization, execution engine, and AI integration in a single coherent system.

Workflow Object

Property / Method	Description
<code>workflow.name</code>	String identifier

<code>workflow.steps</code>	Ordered list of Step objects
<code>workflow.created_at</code>	Timestamp
<code>workflow.last_run</code>	Timestamp of last execution
<code>workflow.save()</code>	Serialize to ~/.serverkit/workflows/
<code>workflow.load()</code>	Deserialize from disk
<code>workflow.export(path)</code>	Export to portable JSON
<code>workflow.run()</code>	Execute all steps in order

Workflow Steps

Each chained method becomes one step node internally:

```
ProcessFilterStep
LogFilterStep
SortStep
SummaryStep
ExportStep
```

Serialization Format

```
{
  "name": "memory_audit",
  "steps": [
    { "type": "process_filter", "memory_above": 1000 },
    { "type": "sort", "field": "memory" },
    { "type": "summary" }
  ]
}
```

Stored at: ~/.serverkit/workflows/memory_audit.json

Execution Output

```
Running workflow: memory_audit

✓ Gathered processes
✓ Filtered memory > 1000MB
✓ Sorted by memory
✓ Generated summary
```

6. Interactive Shell

The shell is a frontend for the SDK — nothing more. It exists for exploration and discovery. It is not an AI chat interface or a second workflow engine.

Launch

```
serverkit    # or  
sk
```

Example Session

```
ServerKit Shell v1.0  
> processes.memory_above(1000)  
> logs('app.log').errors()  
> workflow create memory_audit
```

Deterministic Autocomplete

Autocomplete is NOT AI-powered. It is generated directly from SDK metadata — instant, offline, and deterministic.

```
> proc[TAB] → processes  
  
> processes.[TAB]  
all()          named()          memory_above()  
cpu_above()    sort_by_memory() sort_by_cpu()
```

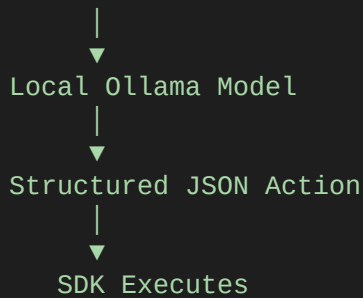
Implementation: `prompt_toolkit` or `readline`

7. AI Layer

The AI layer is optional and strictly constrained. AI translates natural language into SDK actions. AI never executes arbitrary shell commands.

Flow

```
Natural Language
```



Intent → SDK Action

```
server.ask('show python processes using more than 1GB RAM')

# Model returns:
{ "resource": "processes", "action": "memory_above",
  "value": 1000, "filter": "python" }

# SDK executes:
server.processes().named('python').memory_above(1000)
```

AI Workflow Generation

```
server.ask(
    'Create a workflow that checks for processes using
    more than 1GB RAM and summarizes them'
)

# Generates, builds, and saves the workflow automatically
```

Diagnostic Mode

```
server.ask('why is memory usage high?')

# Output:
Memory usage is 91%.
Top consumers:
- python: 2.3 GB
- postgres: 1.1 GB
Python processes appear to be the primary cause.
```


Local Models (Ollama)

Model	Notes
Phi-3 Mini	Preferred — fast, lightweight, accurate for structured output
Qwen2.5 1.5B	Alternative — good JSON generation
SmolLM2	Fallback — minimal resource usage

8. Shared Contracts (Agree Before Branching)

Both developers must agree on these before splitting into separate branches. These are the integration points.

1. Execution Strategy — Eager

Each method executes immediately and returns self. No lazy evaluation.

```
def memory_above(self, mb):
    self.data = [p for p in self.data if p.memory_mb > mb]
    return self
```

2. Return Types

Method	Returns
server.processes()	ProcessCollection (iterable, chainable)
server.logs(path)	LogFile (chainable)
server.workflow(name)	WorkflowBuilder (chainable)
process.details()	dict
.all()	list of objects
.summarize()	str

3. Workflow JSON Schema

```
{ "name": "string",
  "created_at": "ISO8601",
  "steps": [
    { "type": "string", ...params }
  ]
}
```

```
}
```

4. Step Types (Canonical Names)

type	Parameters
<code>process_filter</code>	memory_above, cpu_above, named
<code>sort</code>	field: memory cpu name
<code>log_filter</code>	level: error warning, contains: str
<code>tail</code>	n: int
<code>summary</code>	no params
<code>export</code>	path: str

9. Future Extensions

Module	Example API
<code>Services</code>	<code>server.services()</code> / <code>server.service('nginx').restart()</code>
<code>Docker</code>	<code>server.containers().running()</code>
<code>Disk Analysis</code>	<code>server.disk().largest_files()</code>
<code>Remote Servers</code>	<code>server.connect(host, user)</code>
<code>Workflow Marketplace</code>	<code>server.import_workflow('nginx_health_check')</code>

10. Resume Description

ServerKit – OOP-Based Linux Operations SDK with Local AI Built a Python framework that abstracts Linux processes, logs, and operational workflows into reusable object-oriented components and fluent APIs. Designed an interactive shell with deterministic SDK-aware autocompletion and workflow execution. Integrated a local Ollama-powered AI layer that translates natural language into structured SDK actions and provides server diagnostics while maintaining deterministic execution through the SDK abstraction layer.